

W1 教程：编译链与构建系统

sys-netsec-lab • 阶段一 • 系统基础与 TCP 网关 日期: 2026-07-27 ~ 08-03 教学模式: 先学后做

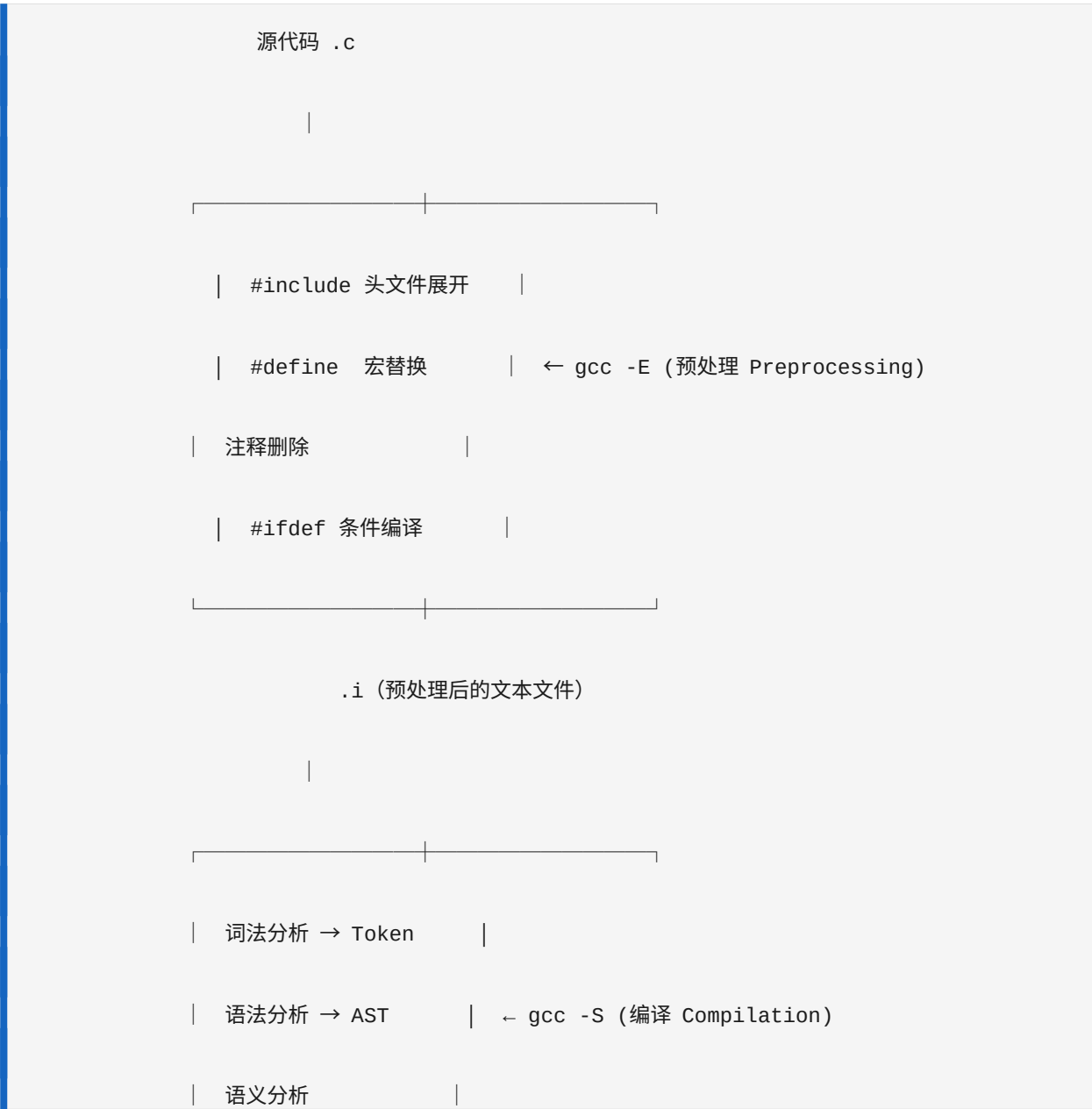
——先理解原理，再动手写代码

目录

1. 第一阶段：理论——编译四阶段
 2. 第二阶段：动手实验（跟我做）
 3. 第三阶段：理解符号表与链接器
 4. 第四阶段：静态库 vs 动态库深度对比
 5. 第五阶段：实战任务
 6. 参考速查表
 7. 常见面试追问
-

第一阶段：理论——编译四阶段

很多开发者把 `gcc main.c -o main` 当成"一步", 但其实 GCC 在背后做了四件事。面试官会追问到每步的细节, 所以你必须能手画这张图:



| 生成汇编代码 |

|-----|

.s (汇编代码文件)

|

|-----|

| 汇编器翻译为机器码 |

| 生成 ELF 目标文件 | ← gcc -c (汇编 Assembly)

| 包含符号表、重定位表 |

|-----|

.o (目标文件, 二进制)

|

|-----|

| 解析所有符号引用 |

| 合并段 (.text .data) | ← ld / gcc (链接 Linking)

| 确定虚拟地址 |

| 生成最终 ELF 可执行 |

|-----|

可执行文件 (ELF 格式)

1.1 预处理 (Preprocessing)

命令： `gcc -E source.c -o source.i`

做了什么：

- `#include` 的文件内容直接插入（所以 .i 文件会比 .c 大很多）
- `#define` 宏被逐字替换
- 注释被删除
- `#ifdef` / `#ifndef` 条件编译被求值

实际例子：

源文件只有 7 行：

```
#include

#define GREETING "Hello"

int main() {

    printf("%s, World\n", GREETING);

    return 0;

}
```

预处理后可能有 2000+ 行——因为 又包含了其他头文件，层层展开。

面试常问："预处理阶段 `#include` 和 `#define` 的区别是什么？"

回答要点：`#include` 是文件级别的文本插入（复制粘贴头文件内容），`#define` 是 token 级别的文本替换（字符串替换）。两者都发生在编译之前。

1.2 编译 (Compilation)

命令：`gcc -S source.i -o source.s`

做了什么：

- 词法分析：把 C 代码拆成 token（关键字、标识符、运算符）
- 语法分析：把 token 构建成 AST（抽象语法树）
- 语义分析：检查类型匹配、作用域等
- 代码生成：输出目标平台的汇编代码

你应该能从汇编代码中认出：

- `push %rbp / mov %rsp, %rbp` —— 函数栈帧的建立
- `call printf` —— 函数调用
- `.string "Hello"` —— 字符串常量在 `.rodata` 段
- 循环被翻译成 `jmp / cmp / je` 组合

1.3 汇编 (Assembly)

命令： `gcc -c source.s -o source.o` (或者直接从 .c: `gcc -c source.c -o source.o`)

做了什么：

- 汇编器把汇编指令翻译成机器码 (x86-64 二进制指令)
- 生成 ELF 格式的目标文件
- 关键：生成符号表 (symbol table) 和重定位表 (relocation table)

输出文件特点：

- `file hello.o` 会输出：ELF 64-bit LSB relocatable
- "relocatable" 意味着：符号地址还是占位符，不能直接执行
- 用 `nm hello.o` 可以看到符号：T (已定义代码)、U (未定义，需要链接时解决)

1.4 链接 (Linking)

命令： `gcc source.o -o program` (gcc 自动调用 ld)

做了什么：

- **符号解析：** 把每个 U (Undefined) 符号找到对应的 T (Text/code) 定义
- **段合并：** 把所有 .o 的 .text 段拼到一起，所有 .data 段拼到一起
- **地址确定：** 给每个符号分配最终的虚拟地址
- **生成最终 ELF：** 加上程序头、段表、动态链接信息等

如果符号找不到 → 报错 `undefined reference to 'xxx'`——这是链接阶段最常见的错误，也是面试必问考点。

第二阶段：动手实验（跟我做）

重要：每行命令手打，不要复制粘贴。手打才能形成肌肉记忆。

实验一：见证编译四阶段

```
# 创建一个最小程序

cat > /tmp/hello.c << 'EOF'

#include

#define GREETING "Hello"

int main() {

    printf("%s, World\n", GREETING);

    return 0;

}

EOF
```

===== Stage 1: 预处理 =====

```
gcc -E /tmp/hello.c -o /tmp/hello.i
```


问：多少行？

```
wc -l /tmp/hello.i
```

答：通常是 2000+ 行，因为 展开后还包含了 、 等

问: GREETING 宏被替换了吗?

```
grep "GREETING" /tmp/hello.i | head -3
```

你会在某行 `printf` 中找到：

```
printf("%s, World\n", "Hello");
```

GREETING 不再存在，被替换成了字符串字面量 "Hello"

问：注释去哪了？

源文件中的所有注释都消失了

===== Stage 2: 编译 =====

```
gcc -S /tmp/hello.i -fverbose-asm -o /tmp/hello.s
```

-fverbose-asm : 在汇编中保留部分 C 源码作为注释，便于对照

找出以下这几部分，看懂它们：

1. 字符串 "Hello" 放在哪个段? → 找 .string

2. main 函数从哪行开始?

→ 找 main:

3. 函数调用 printf 怎么表示? → 找 call

cat /tmp/hello.s

===== Stage 3: 汇编 =====

```
gcc -c /tmp/hello.s -o /tmp/hello.o
```

```
file /tmp/hello.o
```

输出: ELF 64-bit LSB relocatable, x86-64

"relocatable" = 还不能直接执行，符号地址未确定

`nm /tmp/hello.o`

你会看到类似：

U printf ← U = Undefined, 还没找到 printf 的地址



```
000000000000000000 T main      ← T = Text(code), main 定义在这里
```



000000000000000000 r .LC0

← r = 只读数据 (我们的 "Hello, world\n")

000000000000000000 R GREETING ← ? 等等，GREETING 还在吗?

#

GREETING 作为宏在预处理阶段已经被替换了，所以符号表中不会有它。

.LC0 是编译器为字符串字面量生成的匿名标签。

```
objdump -d /tmp/hello.o
```


查看 main 函数的机器码反汇编

===== Stage 4: 链接 =====

```
gcc /tmp/hello.o -o /tmp/hello
```

```
file /tmp/hello
```

输出: ELF 64-bit LSB pie executable, dynamically linked

"executable" = 可以运行了 ✓

"dynamically linked" = 依赖系统的 libc.so

ldd /tmp/hello

你会看到：

linux-vdso.so.1 → 内核提供的虚拟动态库

libc.so.6

→ C 标准库

/lib64/ld-linux-x86-64.so.2 → 动态链接器本身

```
readelf -l /tmp/hello | grep -E "LOAD|INTERP"
```

LOAD 段 = 需要加载到内存的部分（代码、数据）

INTERP = 动态链接器的路径

实验二：亲手制造链接错误

这是最重要的实验。面试一定会问到怎么排查链接错误。

```
# 步骤 1: 声明一个不存在的函数

cat > /tmp/broken.c << 'EOF'

#include

// 声明了 not_here 函数，但整个系统里都没有它的实现

extern int not_here(int x);

int main() {

    printf("Calling not_here...\n");

    int result = not_here(5);

    printf("Result: %d\n", result);

    return 0;

}
```

EOF

步骤 2: 只编译不链接 — 能通过吗?

```
gcc -c /tmp/broken.c -o /tmp/broken.o
```

```
echo "Exit code: $?"
```

通过！编译阶段只看声明是否存在，not_here 有声明（extern int not_here(int)），

编译器不知道（也不关心）它有没有实现。

```
nm /tmp/broken.o | grep not_here
```

输出: U not_here

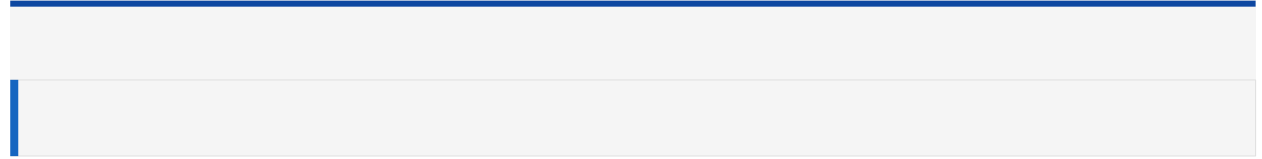
注意这个 U – 编译器标记了"我需要 not_here，但不知道它在哪"

步骤 3: 尝试链接 — 这次会怎样?

```
gcc /tmp/broken.o -o /tmp/broken
```

报错：


```
/usr/bin/ld: /tmp/broken.o: in function main':
```



```
broken.c:(.text+0x1c): undefined reference to not_here'
```

collect2: error: ld returned 1 exit status

这就是链接阶段最经典的错误信息！	

翻译成人类话：链接器 ld 在所有的 .o 和 .a/.so 里都找不到 not_here 的定义

从这个实验学到的关键知识：

阶段	错误类型	错误消息关键字	原因
编译	语法/类型错误	error:	语法不对、类型不匹配、找不到头文件
编译	未声明的标识符	undeclared	没写 #include 或拼写错误
链接	未定义的引用	undefined reference to	有声明但没定义，或没链接库
链接	重复定义	multiple definition of	两个 .o 提供了同名的全局函数/变量

第三阶段：理解符号表与链接器

3.1 nm 命令详解

nm (Name Mangling / Symbol Names) 查看目标文件中的符号表。符号表 = 链接器的地图。

创建一个有各种类型符号的文件

```
cat > /tmp/sydemo.c << 'EOF'
```

```
#include
```

```
int global_var = 42;          // 全局已初始化 → D
```

```
const int const_var = 100;    // 全局常量 → R
```

```
static int hidden_var = 7;     // static 全局 → d (小写=局部可见)
```

```
static void hidden_func() {    // static 函数 → t (小写=局部可见)
```

```
    printf("I am hidden\n");
```

```
}
```

```
void public_func() {          // 非 static 函数 → T
```

```
    printf("I am public\n");
```

```
    hidden_func();            // 调用自己的静态函数
```

```
}
```

```
int main() {
```

```
    public_func();
```

```
    return 0;
```

```
}
```

```
EOF
```

```
gcc -c /tmp/sydemo.c -o /tmp/sydemo.o
```

```
nm /tmp/sydemo.o
```

nm 输出解释:

标识	含义	关键理解
T	Text (code) — 全局函数	链接器可以跨文件找到它
t	Local text — static 函数	只在当前 .o 内可见，链接器看不到
D	Data — 已初始化全局变量	.data 段
d	Local data — static 变量	只在当前文件可见
B	BSS — 未初始化全局变量	.bss 段（程序启动时清零，不占文件空间）
R	Read-only data — 常量	.rodata 段
U	Undefined — 未定义	需要链接器去别的文件或库里找

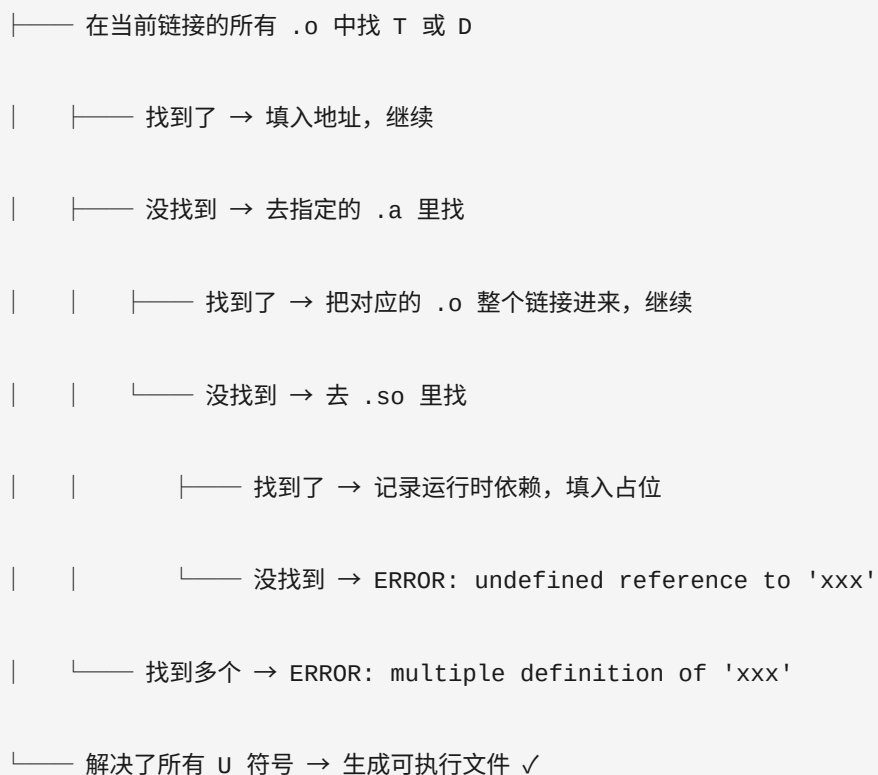
面试关键问题: "static 关键字对链接的影响是什么? "

回答: static 函数/变量的符号是小写 (t/d) , 只在本 .o 内可见。链接器看不到它们，所以不会和其他文件的同名符号冲突。这就是 C 语言实现"模块私有"的方式——通过限制符号可见性。

3.2 链接器的工作流程

链接器 ld 的决策树：

对每个 .o 文件中的 U 符号：



3.3 验证链接器的行为

查看 printf 的符号在哪里被定义

```
nm /usr/lib/x86_64-linux-gnu/libc.so 2>/dev/null | grep -w printf
```

```
nm /usr/lib/x86_64-linux-gnu/libc.a 2>/dev/null | grep -w printf | head -3
```

会发现 printf 的 T 定义在 libc 里

查看你的程序的完整链接过程

```
gcc /tmp/sydemo.o -o /tmp/sydemo -Wl,--trace 2>&1
```

```
-Wl,--trace 让链接器打印它加载了哪些文件
```

第四阶段：静态库 vs 动态库深度对比

4.1 静态库 .a

本质： `.a` = 一组 `.o` 文件的 `tar` 包。`ar` 命令就是归档工具。

构建步骤：

```
# 1. 正常编译每个 .c 成 .o

gcc -c add.c -o add.o

gcc -c mul.c -o mul.o
```

2. 用 ar 打包

```
ar rcs libmath.a add.o mul.o
```


r = 插入文件（替换同名旧文件）

c = 创建归档文件（如果不存在）

s = 写入索引（符号→.o 的映射，加速链接）

3. 验证

```
ar t libmath.a      # 列出成员  
  
nm libmath.a        # 查看所有符号
```

链接时的行为：

```
gcc main.c -L. -lmath -o prog
```

链接器做的事：

1. 看到 main.c 中有 add 和 mul 的 U 符号

2. 去 libmath.a 的索引中查找

3. 找到 add 在 add.o 中 → 把整个 add.o 链接进来

4. 找到 mul 在 mul.o 中 → 把整个 mul.o 链接进来

关键事实：静态链接时，链接器只从 .a 中**按需提取**需要的 .o，不是把整个 .a 塞进去。每个被引用的符号所在的 .o 才会被链接。

优缺点：

优点	缺点
可执行文件独立，不需要额外的库文件	可执行文件体积大（每个程序都有一份库代码的拷贝）
无版本兼容问题（编译时确定）	库更新后必须重新编译链接
启动速度稍快（不需要运行时加载）	多个程序不能共享内存中的同一份库代码

4.2 动态库 .so

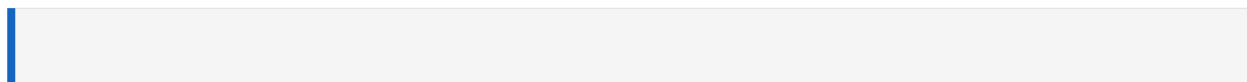
本质： .so = 位置无关的、可直接加载到内存执行的代码。

构建步骤：

```
# 1. 编译时加 -fPIC (Position Independent Code)

gcc -fPIC -c add.c -o add_pic.o

gcc -fPIC -c mul.c -o mul_pic.o
```



2. 链接成动态库，设置 soname（运行时标识）

```
gcc -shared -Wl,-soname,libmath.so.1 -o libmath.so.1.0.0 add_pic.o mul_pic.o
```

3. 创建符号链接（遵循 Linux 共享库命名规范）

```
ln -sf libmath.so.1.0.0 libmath.so.1      # soname → 实际文件

ln -sf libmath.so.1 libmath.so            # linker name → soname
```

soname 命名规范（重要面试话题）：

```
libcalc.so.1.2.3

|   |   |   | └─ release (bugfix)

|   |   | └─── minor (新功能+向后兼容)

|   | └────── major (不兼容变更)

| └────────── 动态库扩展名

└────────── 库名前缀
```

规则：major 变了 = 不兼容，程序必须用匹配的 major 版本

运行时搜索路径的优先级：

1. 可执行文件内嵌的 RPATH / RUNPATH (\$ORIGIN 常用)

`LD_LIBRARY_PATH` 环境变量

`/etc/ld.so.cache` (由 `ldconfig` 维护)

系统默认路径 `/lib`, `/usr/lib`

\$ORIGIN 是什么?

`$ORIGIN` = 可执行文件所在的目录

这对于打包发布非常重要——不用硬编码绝对路径

```
gcc main.c -L. -lmath -Wl,-rpath,'$ORIGIN/../lib' -o myapp
```

运行时：

./myapp → 去 myapp 所在目录/./lib 找 libmath.so

无论 myapp 被移动到哪个目录，都能正确找到库

优缺点：

优点	缺点
可执行文件小	需要 .so 文件随程序分发
多个进程共享同一份 .so 内存页	版本不匹配会导致运行失败
更新 .so 无需重新编译程序	启动时需要动态链接器解析符号（微小开销）
插件架构的基础（dlopen）	依赖地狱（A 要 v1，B 要 v2）

4.3 对比速查表

维度	静态库 .a	动态库 .so
本质	.o 文件的归档（tar 包）	位置无关的可执行代码
创建命令	<code>ar rcs libxx.a .o</code>	<code>gcc -shared -fPIC -o libxx.so .o</code>
链接方式	<code>gcc main.c libxx.a</code>	<code>gcc main.c -L. -lxx</code>
链接时行为	需要的 .o 嵌入可执行文件	只记录运行时依赖

可执行文件大小	大	小
运行时依赖	不需要	需要 .so 在搜索路径中
查看符号	<code>nm libxx.a</code>	<code>objdump -T libxx.so</code>
库更新后	需要重新链接	通常不需要（接口兼容的前提下）
强制选择	<code>-Wl,-Bstatic -lxx -Wl,-Bdynamic</code>	默认优先 .so，除非强制静态
内存共享	每个进程一份	多进程共享同一物理页

面试要点：当目录下同时有 `libcalc.a` 和 `libcalc.so` 时，`gcc -lcalc` 默认选 `.so`。要强制用 `.a`，需要 `-Wl,-Bstatic -lcalc -Wl,-Bdynamic`。

第五阶段：实战任务

现在你有足够的知识了，开始写代码。

任务一：libcalc 计算库（编码 A，~3h）

规格：

写一个 C 库，实现以下功能：

```
// calc.h 中的公开接口

int calc_add(int a, int b);          // 加法

int calc_sub(int a, int b);          // 减法

int calc_mul(int a, int b);          // 乘法

int calc_div(int a, int b, int *result); // 除法：成功返回 0，除零返回 -1

int calc_pow(int base, int exp);      // 整数幂运算

int calc_get_count(void);             // 返回累计运算次数

void calc_reset(void);               // 重置计数器

const char* calc_version(void);      // 返回版本字符串
```

要求：

1. `calc.h` 和 `calc.c` 分离
2. 计数器用 `static` 变量实现（验证符号表中的小写 `d`）
3. 用辅助 `static` 函数实现计数递增（验证 `t`）
4. 写 Makefile，两个 target：

- `libcalc.a` — 用 `ar rcs`

- `libcalc.so` — 带 soname 版本号，创建正确的符号链接

1. 构建后用 `nm` 验证 static 函数不导出

任务二：链接对比演示（编码 B, ~3h）

要求：

1. 写 `demo/main.c`，调用 `libcalc` 的所有函数
2. 写 Makefile，两个 target：

- `demo-static`：用 `-Wl,-Bstatic -lcalc -Wl,-Bdynamic` 强制链接 `.a`

- `demo-shared`：链接 `.so`，用 `$ORIGIN` 设置 `rpath`

1. 对比两种链接方式：

- 文件大小差异

- `ldd` 显示的依赖差异

- `readelf -d` 的 `NEEDED` 差异

- `nm` 能否在可执行文件中找到 `calc_add`

任务三：编译四阶段笔记（实验，~2.5h）

要求：

- 1. 写一个小C程序（至少包含宏、全局变量、函数调用）
- 2. 逐步执行四个阶段，记录每个阶段的观察：

- .i：行数变化、宏展开结果

- .s：函数序言、调用序列

- .o：nm 符号类型

- 可执行文件：ldd、readelf -l

参考速查表

常用命令一览

目的	命令	说明
预处理	<code>gcc -E src.c -o src.i</code>	宏展开，头文件展开

生成汇编	<code>gcc -S src.c -fverbose-asm -o src.s</code>	-fverbose-asm 添加源码注释
只编译不链接	<code>gcc -c src.c -o src.o</code>	生成目标文件
完整编译	<code>gcc src.c -o program</code>	四阶段一步完成
创建静态库	<code>ar rcs libxx.a a.o b.o</code>	r=插入 c=创建 s=索引
创建动态库	<code>gcc -shared -fPIC -Wl, -soname,libxx.so.1 -o libxx.so.1 a.o</code>	别忘 -fPIC
强制静态链接	<code>gcc main.c -Wl,-Bstatic -lxx -Wl,-Bdynamic</code>	夹在中间
设置 rpath	<code>gcc main.c -Wl,-rpath,'\$ORIGIN/../lib'</code>	\$ORIGIN=可执行文件目录
列出 .a 成员	<code>ar t libxx.a</code>	看里面打了哪些 .o
看 .o 符号	<code>nm file.o</code>	T/D/B/U + 地址
看 .so 导出符号	<code>objdump -T libxx.so</code>	动态符号表
看链接依赖	<code>ldd program</code>	运行时加载哪些 .so
看 ELF 段	<code>readelf -l program</code>	LOAD 段布局
看 ELF 段头	<code>readelf -S file.o</code>	.text .data .bss .rodata

看 RUNPATH	<code>readelf -d program \</code>	<code>grep -i path</code>	动态库搜索路径
反汇编	<code>objdump -d file.o</code>	机器码→汇编码	
链接器详细日志	<code>gcc ... -Wl,--trace</code>	看链接器加载了哪些文件	
动态库搜索路径	<code>ldconfig -p \</code>	<code>grep libname</code>	查看系统缓存的 .so

错误消息速查

错误消息	阶段	原因	解决
<code>fatal error: xxx.h: No such file or directory</code>	预处理	头文件路径不对	<code>-I/path/to/include</code>
<code>error: 'xxx' undeclared</code>	编译	没声明，头文件缺失或拼写错误	检查 #include
<code>error: expected ';' before ...</code>	编译	语法错误	检查上一行
<code>undefined reference to 'xxx'</code>	链接	有声明无定义，或没链接库	<code>-l</code> 指定库，或检查函数名拼写
<code>multiple definition of 'xxx'</code>	链接	两个 .o 都定义了同名的	加 static 或用 inline

非 static 函数

<code>cannot open shared object file</code>	运行时	.so 不在搜索路径中	设 LD_LIBRARY_PATH 或 rpath
<code>version 'GLIBC_2.xx' not found</code>	运行时	glibc 版本不够	目标系统 glibc 太旧

常见面试追问

以下是面试官可能会基于这周内容追问的问题，提前准备：

1. "预处理阶段做了哪些事？"（高频）

标准回答：`#include` 文件展开、`#define` 宏替换、条件编译处理、注释删除。预处理后得到的依然是文本文件。可以用 `gcc -E` 验证。

2. "`static` 在 C 中有哪两种用法？分别影响什么？"（高频）

标准回答：

- **static 全局变量/函数**：限制作用域在文件内，阻止符号导出到链接器。用 `nm` 验证是大写 `T/D` 还是小写 `t/d`。
- **static 局部变量**：生命周期变为全局（函数退出后不销毁），但作用域仍限于函数内。存储在 `.data` 段而非栈上。

3. "如果链接时既有 `libxx.a` 又有 `libxx.so`，链接器选哪个？怎么强制选 `.a`？"

标准回答：默认选 `.so`。用 `-Wl,-Bstatic -lxx -Wl,-Bdynamic` 强制选 `.a`。

4. "动态库的 `soname` 是什么？为什么需要它？"

标准回答：`soname` 是动态库的**运行时逻辑名称**（如 `libcalc.so.1`）。它允许库的 major 版本锁定——多个 minor/patch 版本可以共享同一个 `soname`，升级小版本不需要重新链接程序。实际文件名可能是 `libcalc.so.1.2.3`，但运行时按 `soname libcalc.so.1` 查找。

5. "为什么改了 `.c` 文件要重新编译，但只改 `.so` 文件不需要？"

标准回答：改 `.c` → `.o` 变了 → 可执行文件里的静态链接代码是旧的，必须重新链接。改 `.so` → 可执行文件里只记录了 `.so` 的 `soname` 和符号名，运行时动态链接器去加载最新的 `.so`，所以只要接口兼容就能用新版本。

6. "\$ORIGIN 是什么，为什么推荐用它？"（中频）

标准回答：`$ORIGIN` 是 ELF 中的一个特殊标记，指向可执行文件所在的目录。用法 `-Wl,-rpath,'$ORIGIN/../lib'`。好处是程序可以整体移动，不用硬编码绝对路径，适合打包发布。

验收清单

完成实验和任务后，逐项检查：

- ☐ 删除所有 `.o .a .so` 后，一行 `make` 能全部重建
 - ☐ `nm libcalc.a` 能看到 `calc_add` 标记为 `T`，`static` 辅助函数标记为 `t`
 - ☐ `demo-static` 运行正常，`ldd` 不依赖 `libcalc.so`
 - ☐ `demo-shared` 运行正常，`readelf -d` 显示 `NEEDED: libcalc.so.1`
 - ☐ 能对着 `.i` 文件指出宏展开前后的变化
 - ☐ 能解释 `undefined reference to` 错误发生在哪个阶段
 - ☐ 能区分 `nm` 输出中 `T`、`t`、`U`、`D`、`d` 的含义
 - ☐ 能手画编译四阶段流程图，标注每阶段输入输出
-

>  **下一周**：W2 — C 内存与调试（5 类内存缺陷、ASan/UBSan/Valgrind）

>

>  本教程由 Claude Code 作为技术导师编写 • sys-netsec-lab